Binary Search Trees (BST)

The Ultimate Guide to Operations and Efficiency

1. Understanding Binary Search Trees (BST)

A Binary Search Tree (BST) is a node-based binary tree data structure which has the following properties:

- The **left subtree** of a node contains only nodes with keys lesser than the node's key.
- The **right subtree** of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.

Pro-Tip: An In-order traversal of a BST always yields the keys in non-decreasing (sorted) order.

2. Searching in a BST

Searching is highly efficient. We compare the target value with the root; if it's smaller, we move left; if larger, we move right. This effectively halves the search space at each step in a balanced tree.

```
def search(node, target):
    if node is None or node.data == target:
        return node
    if target < node.data:
        return search(node.left, target)
```

```
    return search(node.right, target)
```

3. Insertion and Min-Value Logic

To insert, we traverse until we find a None spot that satisfies the BST property. Additionally, finding the minimum value in a subtree is crucial for deletion, achieved by following the leftmost path.

```
def insert(node, data):
    if node is None:
        return TreeNode(data)
    if data < node.data:
        node.left = insert(node.left, data)
    else:
        node.right = insert(node.right, data)
    return node

def minValueNode(node):
    current = node
    while current.left:
        current = current.left
    return current
```

4. The Deletion Process

Deletion handles three scenarios:

1. **Leaf Node:** Simply remove the link.
2. **One Child:** Replace the node with its child.
3. **Two Children:** Find the in-order successor (smallest in right subtree), replace the node's value with the successor's value, and delete the successor.

5. Complexity & Balance

Performance depends entirely on the tree's height *h*.

Scenario	Time Complexity
Balanced Tree	O(log n)
Unbalanced (Skewed)	O(n)

Balanced trees (like AVL or Red-Black trees) ensure height is kept at log(n) for optimal performance.

Case Study: Successor and Predecessor (Part 1)

In a BST, finding the 'In-order Successor' is vital for the deletion of nodes with two children. The successor is the node with the smallest key greater than the key of the input node.

Logic: If the right subtree is not empty, the successor is the `minValueNode` of the right subtree. If it is empty, the successor is one of the ancestors.

Why BST over Arrays?

While an array allows O(log n) search via Binary Search, inserting a new element requires O(n) because other elements must be shifted. A BST allows O(log n) insertion without any pointer-shifting besides the local link update, making it superior for dynamic datasets.

Case Study: Successor and Predecessor (Part 2)

In a BST, finding the 'In-order Successor' is vital for the deletion of nodes with two children. The successor is the node with the smallest key greater than the key of the input node.

Logic: If the right subtree is not empty, the successor is the `minValueNode` of the right subtree. If it is empty, the successor is one of the ancestors.

Why BST over Arrays?

While an array allows O(log n) search via Binary Search, inserting a new element requires O(n) because other elements must be shifted. A BST allows O(log n) insertion without any pointer-shifting besides the local link update, making it superior for dynamic datasets.

Case Study: Successor and Predecessor (Part 3)

In a BST, finding the 'In-order Successor' is vital for the deletion of nodes with two children. The successor is the node with the smallest key greater than the key of the input node.

Logic: If the right subtree is not empty, the successor is the `minValueNode` of the right subtree. If it is empty, the successor is one of the ancestors.

Why BST over Arrays?

While an array allows O(log n) search via Binary Search, inserting a new element requires O(n) because other elements must be shifted. A BST allows O(log n) insertion without any pointer-shifting besides the local link update, making it superior for dynamic datasets.